4. Project Design Phase

4.1 Introduction

The design phase transforms the requirements into a structured system design that defines the components, data flow, and functionality of EduGenie. The system follows a modular architecture consisting of a user interface, an application server, a database, and a Gemini AI integration module that communicate through well-defined APIs.

4.2 System Design

EduGenie follows a client-server architecture. The user interacts with the frontend, which sends requests to the FastAPI backend through REST APIs. The backend validates requests, communicates with the database, builds prompts, calls the Google Gemini API, processes the response, and returns it to the frontend. The Gemini API key is kept only on the server.

4.3 System Architecture

- Presentation Layer: login, registration, dashboard, tutor chat, summarizer, quiz, flashcards, study plan, and history pages.
- Application Layer: Python and FastAPI handling authentication, request validation, prompt construction, and business logic.
- AI Layer: Google Gemini model that generates explanations, summaries, quizzes, flashcards, and study plans.
- Data Layer: SQLite or PostgreSQL storing users, chat history, summaries, quizzes, scores, and study plans.

4.4 System Architecture Flow

User → Frontend → FastAPI Backend → Prompt Builder → Gemini API → Backend → Database → Frontend → User

4.5 Module Design

Module	Design Description
Authentication	Registration, login with password hashing, JWT token issuing and verification, profile management, logout.
AI Tutor Chat	Receives question, subject, and level; sends conversation context to Gemini; returns and stores the answer.
Concept Explanation	Builds a level-specific prompt requesting explanation, examples, and key points.
Notes Summarization	Accepts text, requests summary and key takeaways from Gemini, stores the result.
Quiz	Requests structured questions (JSON) from Gemini, evaluates submitted answers, stores the score.
Flashcards	Generates term–definition pairs from a topic or notes for flip-style review.
Study Plan	Takes subject, topics, time per day, and target date; returns a structured schedule.

Module	Design Description
History & Dashboard	Retrieves saved items and aggregates scores and activity for charts.

4.6 Database Design

Table	Main Fields
users	id, name, email, password_hash, created_at
chat_history	id, user_id, subject, question, answer, created_at
summaries	id, user_id, original_text, summary, created_at
quizzes	id, user_id, topic, difficulty, questions_json, score, created_at
flashcards	id, user_id, topic, front, back
study_plans	id, user_id, subject, target_date, plan_json, created_at

4.7 API Design

Method	Endpoint	Purpose
POST	/auth/register	Create a new user account
POST	/auth/login	Authenticate and return JWT
POST	/chat	Ask the AI tutor a question
POST	/explain	Explain a topic at a chosen level
POST	/summarize	Summarize supplied notes
POST	/quiz/generate	Generate a quiz on a topic
POST	/quiz/submit	Evaluate answers and save the score
POST	/flashcards	Generate flashcards
POST	/study-plan	Generate a study plan
GET	/history	Retrieve the user's saved items
GET	/dashboard	Retrieve progress statistics

4.8 Security Design

- Passwords are stored as hashes, never as plain text.
- JWT tokens protect all private endpoints.
- The Gemini API key is stored in environment variables on the server.
- User input is validated and sanitized before being sent to the AI model.